STM32 小车开源代码使用说明

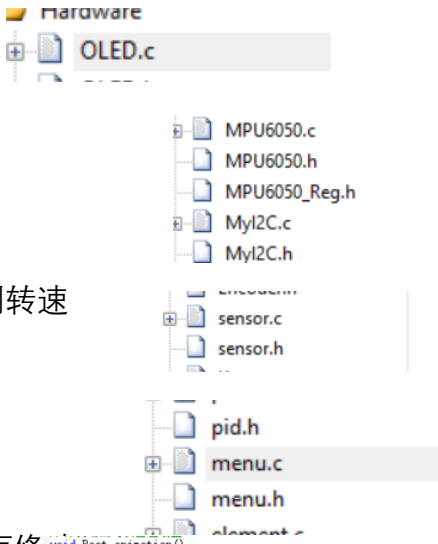
作者：b 站芋头雨-　　日期：2025.5.27

声明：

此代码参考了江科大的部分模块代码，龙邱科技的差速部分，欢迎大家随意修改学习传播，但不得作为商业用途！

一、 整体框架

基础模块搭建：



1、 oled 屏幕显示：用于循迹调参、状态位显示

2、 i2c 以及陀螺仪：用于辅助转向以及元素处理

3、 编码器、pwm：用于控制轮子输出 pwm 闭环控制转速

4、 传感器：及灰度传感器，用于识别黑线，循迹

5、 其他：串口、按键：辅助观察参数、闭环辅助等

6、 菜单：很基础的菜单，大家只需要在这些函数里面修改参数即可，其他无需修改。(按键引脚要改一下，不然无法翻页)

```
void Boot_animation()
{
    OLED_ShowString(40,32,"Hello!!!",OLED_8X16);
}

//第一页
void Homepage_1()
{
        OLED_Printf(0,1,OLED_8X16," Speed_L:%+05d",Speed_L);
        OLED_Printf(0,17,OLED_8X16,"Speed_R:%+05d",Speed_R);
        OLED_Printf(0,34,OLED_8X16,"PWM_L:%+02d",Speed_Out_L);
        OLED_Printf(0,51,OLED_8X16,"PWM_R:%+02d",Speed_Out_R);
        OLED_Update();
}
//第一页
void Homepage_2()
{
        OLED_Printf(0,0,OLED_8X16," s_err:%+04d",sensor_err);
        OLED_Printf(0,17,OLED_8X16,"f_err:%+04d",final_err);
        OLED_Printf(0,34,OLED_8X16,"P_Out:%+04d",Place_Out);
        OLED_Printf(0,51,OLED_8X16,"gz:%+03d",GZ);
        OLED_Update();
}
//第三页
void Homepage_3()
{
            OLED_Printf(0,0,OLED_8X16," s:%+06.2f",Location);
            OLED_Printf(0,17,OLED_8X16,"yaw:%+03.2f",yaw);
            OLED_Printf(0,34,OLED_8X16,"FE:%+02d",Element_Flag);
            OLED_Printf(0,51,OLED_8X16,"FR:%+02d",Ring_Flag);
}
//第四页
void Homepage_4()
{
            OLED_Printf(0,0,OLED_8X16," FP:%+03d",Place_Enable);
            OLED_Printf(0,17,OLED_8X16,"FN:%+02d",Noline_Flag);
            OLED_Printf(0,34,OLED_8X16,"FT:%+02d",Ten_Flag);
            OLED_Printf(0,51,OLED_8X16,"FS:%+02d",Stop_Flag);
```

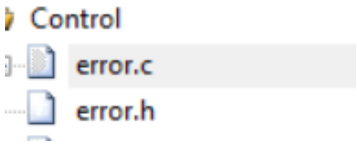
参考教程：[STM32 入门教程-2023 版 细致讲解 中文字幕_哔](#)

陀螺仪：10-3 软件读取 mpu6050

[PID 入门教程-电机控制 倒立摆 全程手把手打代码调试_哔哩哔哩_bilibili](#)

基础驱动编写以及调试：1-1~1-3

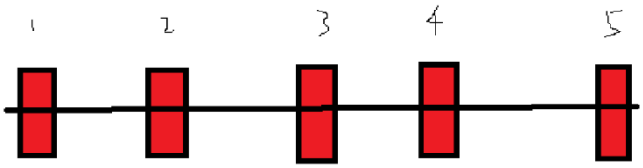
二、 控制部分



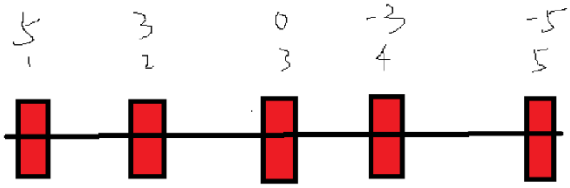
1、循迹：

原理：五路传感器默认低电平，灯亮，黑线无反射，灯灭。我们给五路传感器分别乘上不同的值，以此代表偏离直线的多少。

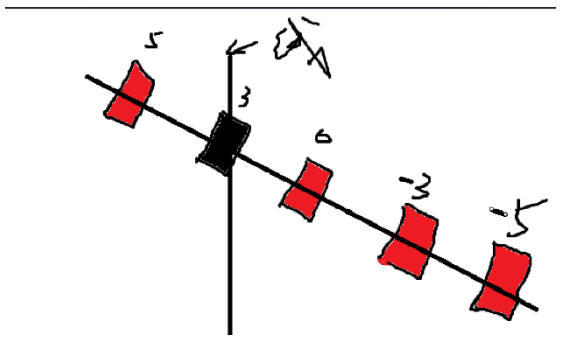
例如：



加权重：



有偏移：



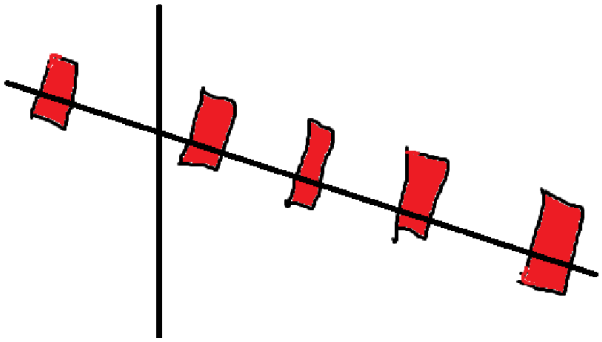
这时：如果我们使用一个 err=5*L2+3*L1+0*M-3*R1-5*R2

那我们就可以得到一个随位置变化而变化的 err 值了

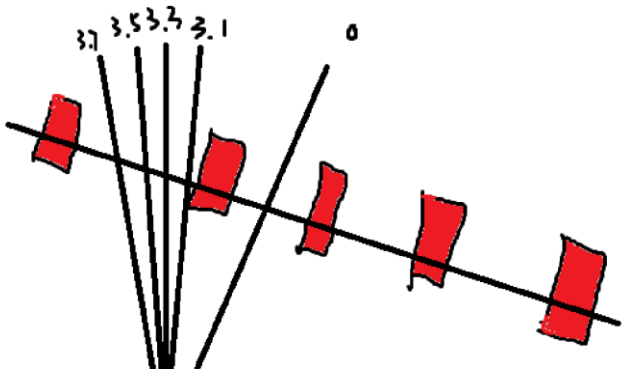
```
// 根据传感器物理位置定义坐标（假设线性排列）
static const SensorMap sensorMap[SENSOR_NUM] =   //数组是一个常量数组，且只能在定义它的文件或代码段内访问。const 表明数组中的元素值不可修改
{
    {-2.8, 1}, // L2
    {-1.3, 2}, // L1
    { 0, 3}, // M
    { 1.3, 4}, // R1
    { 2.8, 5}  // R2
};

/* ***************计算传感器偏差 *************** */
int Error_Calcaulate()
{
    int active_sum = 0;
    int active_count = 0;
     for (int i = 0; i < SENSOR_NUM; i++)
    {
            if (digital(sensorMap[i].channel))          //digital 的函数，传入传感器的通道号。
                {
                active_sum += sensorMap[i].position; // 累加激活传感器的位置
                active_count++;                                    //在循环中，active_count 会在每个激活的传感器时增加 1;
                    }
   }

                err = active_count ? (active_sum * SCALE_FACTOR) / active_count : 0;    //这里是最终输出err后然后限幅，只是比较复杂的写法
                err = err*Right_err();              //直角偏差放大
    return err;
}
```

也就是这一段看似复杂的代码实际上在干的事情。只不过用了遍历和结构体。

2、陀螺仪辅助循迹



我们在实际调节过程中可能会发现在传感器之间存在一些空隙导致无法正常控制，如果给他保存上一个值的话就出现了 5-3-0-3-5 这样阶跃的过程，也就是在 5-3 不是线性变化的，这样会导致控制不够丝滑。最后我发现用陀螺仪可以解决这个问题。



我们假设车身中间为 0°，只要我们通过转换，使得在两个传感器之间的值是我们期望的，3.1，3.3，3.5，3.7 这样。而陀螺仪的变化又是线性的，那不就弥补了这空缺无法线性变化的缺点了吗。

于是我们就可以得到 5-3.7-3.5-3.3……3.1-0 这样的 err。这样的 err 看起来就舒服多了。

所以：先做积分处理，并根据期望的值做出相应的放大减小。

```
/**** 步骤1：陀螺仪积分处理 ****/

float delta_err = gyro_z*GYRO_SCALE;// 将角速度放大

// 积分计算（假设调用周期为dt秒）
float dt = 0.01f; // 1ms周期（我的实际周期是5ms这里我希望他积分快一点）
fstate.integrated_err += delta_err * dt;
```

在没有传感器的时候我们就用陀螺仪：

```
        // 情况B：无传感器信号时，信任陀螺仪
else
{
        fused_err= fstate.integrated_err;          //没有传感器的时候，陀螺仪为不断积分，误差一直变大
}
```

另外：MPU6050 存在积分有误差的现象，因此我们每次有传感器经过时给他重置一下积分，这样积分的误差就在可接受的范围内被不断刷新。

```
if( sensor_err!=0||digital(3)==1)
    {
            // 情况A：有传感器信号时，重置积分，重置方式为0.4的传感器0.6的陀螺仪。
    fstate.integrated_err = fstate.integrated_err * 0.6f + sensor_err * 0.4f;
```

在有传感器的时候：

```
fstate.last_sensor_err = sensor_err;
fused_err=(1-FUSION_ALPHA)* fstate.integrated_err + FUSION_ALPHA*sensor_err;
```

我们用一部分传感器算出来的误差一部分陀螺仪算出来的误差，也就是互补滤波，通过调节系数可以到一个很好的状态。

```
/* ***************** 陀螺仪辅助计算 ***************** */
int get_fused_error(int sensor_err, float gyro_z)
{
    /**** 步骤1：陀螺仪积分处理 ****/

    float delta_err = gyro_z*GYRO_SCALE;// 将角速度放大

    // 积分计算（假设调用周期为dt秒）
    float dt = 0.01f; // 1ms周期（我的实际周期是5ms这里我希望他积分快一点）
    fstate.integrated_err += delta_err * dt;

    /**** 步骤2：互补滤波融合 ****/
    int fused_err = 0;
    if( sensor_err!=0||digital(3)==1)
        {
            // 情况A：有传感器信号时，重置积分，重置方式为0.4的传感器0.6的陀螺仪。
        fstate.integrated_err = fstate.integrated_err * 0.6f + sensor_err * 0.4f;
//      fstate.last_sensor_err = sensor_err;
        fused_err=(1-FUSION_ALPHA)* fstate.integrated_err + FUSION_ALPHA*sensor_err;
    }

            // 情况B：无传感器信号时，信任陀螺仪
        else
        {
            fused_err= fstate.integrated_err;          //没有传感器的时候，陀螺仪为不断积分，误差一直变大
        }
        /**** 步骤4：限幅处理 ****/
            //和if的算法是一样的只是这样写更简便
            fused_err = (fused_err > SENSOR_MAX_ERR) ? SENSOR_MAX_ERR : ((fused_err < -SENSOR_MAX_ERR) ? -SENSOR_MAX_ERR : fused_err);

    return (int)fused_err;
}
```

偏差值

3、控制（转向环加速度环）

我们用的是串级，转向环的值会传递给速度环。所以我们需要先把速度环调好。

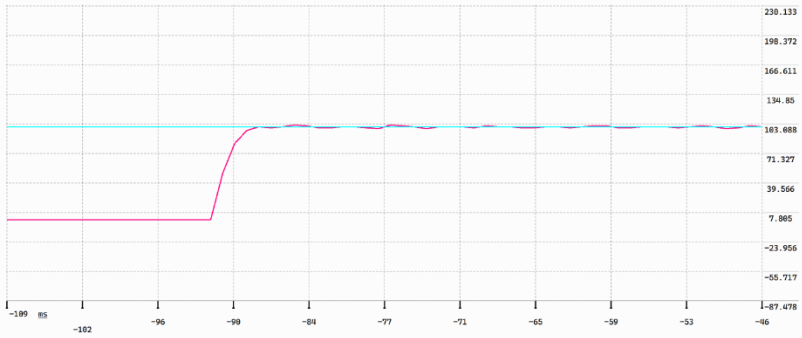
```
/**********************位置式速度环pd*************************/
int PID_Control(int NowPoint, int SetPoint, int *TURN_PID) //PI控制速度环
{
      static int Integral,LastError = 0;      // 积分累积量（静态变量）
        int KP,KI,KD,Out,NowError;
        KP = *TURN_PID;
        KI = *(TURN_PID+1);
        KD = *(TURN_PID+2);

    NowError = SetPoint - NowPoint;

//    // 积分项累加（需限幅防饱和）
//    Integral += NowError;
//    Integral = Min_Max(Integral, -INTEGRAL_MAX, INTEGRAL_MAX); // 示例：INTEGRAL_MAX=1000

        //这里pi、pd我都试过了pd效果更好一点
    Out = KP * NowError + KI * Integral+KD *(NowError-LastError);

    return Out;
}
```

```
int Left_Speed,Right_Speed=0;
int Speed_PID[3] = {250,0,30};
float Place_PD[2] = {5.3,4.3};
```

我们可以参考江科大的 pid 调参教程，将实际速度输入到

```
menu_operation();
Serial_Printf("%d,%d,%f,%f\r\n", 1,Speed_R,yaw,Location); //串口输出（
OLED_Update();
```

这样我们能通过串口显示的方式调出一个响应即使且不超调的图像



速度环调好之后，我们将转向环和速度环串在一起，

```
//转向环
if(Place_Enable)
{
    Place_Out=(int)Place_Control(final_err,0,Place_PD);
}
//速度环
Speed_Out_L=PID_Control(Speed_L,Left_Speed,Speed_PID);
Speed_Out_R=PID_Control(Speed_R,Right_Speed,Speed_PID);

//输出限幅
```

具体调试以及操作参考:

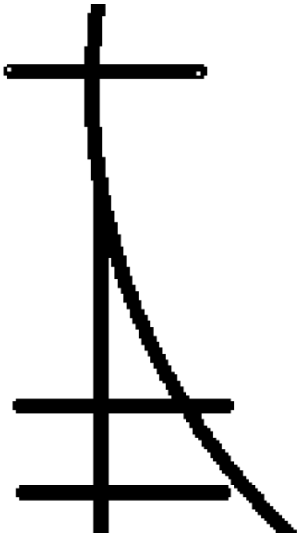
[PID 入门教程-电机控制 倒立摆 全程手把手打代码调试_哔哩哔哩_bilibili]

1-4 集

4、关于差速计算（参考龙邱科技的开源方案）

```
/************************差速计算***************************/
void Different_Speed()
{
 float k;
 //转向环的输出
 //输出的速度取决于基础速度的大小，所以速度快了之后转向力度也会跟着变大
 if(Place_Out >= 0)
     {
         k = Place_Out * 0.01;
         Left_Speed = Basic_Speed * (1 - k);
         Right_Speed = Basic_Speed * (1 + k*Turn_factor);
     }
     else
    {
     k = -Place_Out * 0.01;
     Left_Speed = Basic_Speed * (1 + k*Turn_factor);
     Right_Speed = Basic_Speed * (1 - k);
  }
}
```

将转向环输出的值转换成 0~1 之间的数并×上目标速度，那么转向的力度会随着基础速度提升而增加，pid 的适应性会增强。

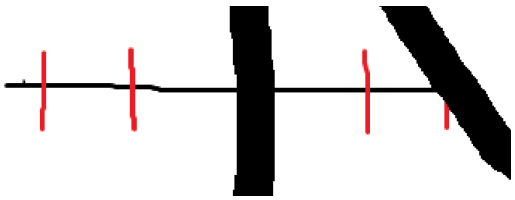


三、 元素部分

1、 以圆环和断路为例:

圆环:

我们放大这个局部元素，就会发现从进入圆环前到进入圆环中到进入圆环后灯有不同的状态，根据不同的状态我们可以识别圆环。

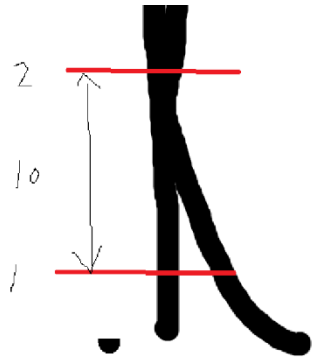


这里我们就做第一次识别

```
        //第一次检测入环
        if(R2==1&&M==1&&(L1==0||R1==0)&&L2==0)//初次检测
    {
            Ring_Flag=1;
            Clear_Location();
    }
```

当识别后，编码器计距 10cm，然后再做判断

```
    //第一次检测到后过10cm再进行检测然后入环
if(Ring_Flag==1&&Location>=10.5)
{
    if((L1==0&&M==1&&R1==0)||((L1==1||R1==1)&&M==1))
    {
```



此时如果右边全亮，中间灭就正式进入圆环，也就

的时候刚好入环。

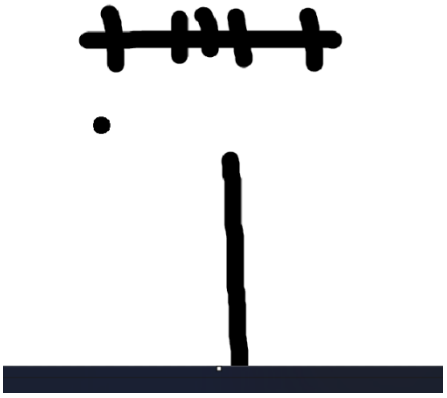
入环我们关掉转向环，给一个固定的差速，这样可以走的快的同时避免其他干扰。

```
Ring_Flag=2;
Clear_Location();
Element_Flag=1;
Place_Enable=0;//关掉转向环
Place_Out=-15; //给一个固定的差速
```

当编码器距离大于 30cm，也就是正式入环，就恢复正常寻线，由于圆环难度比较小，这里我还把他速度拉满了。

```
}
    //结束入环动作，正常巡线
if(Ring_Flag==2&&Location>30)
{
    Place_Enable=1;
    Ring_Flag=3;
    Basic_Speed=Speed_Choice[2];

}
```

出环也是同理。



2、 丢线:

当五个灯全灭的时候就是断路的状态，

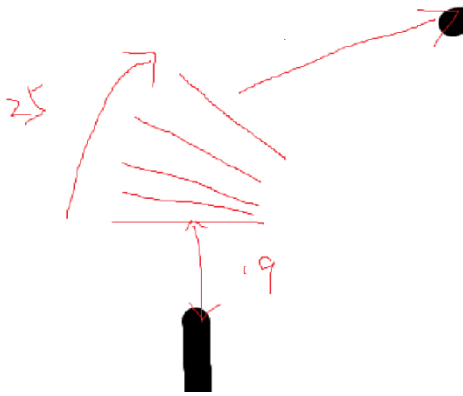
但是很容易误判，所以我们要加一点条件限制。

```
void Element_Noline()
{
        if (Element_Flag == 0 &&Noline_Flag==0)
        {
            if (L2 == 0 && L1 == 0 && M == 0 && R1 == 0 && R2 == 0)  //丢线的原理就是所有灯熄灭
          {
                Noline_Flag = 1;
                Clear_Location();                    //当初次识别丢线之后开始计算路程
          }
    }
    if(Noline_Flag==1)
    {
          if(L2==0&&L1==0&&M==0&&R1==0&&R2==0)
           {
              if(Location>19)                        //当累计路程大于19，正式进入丢线，否则算作误判，不进入丢线
```

也就是灭一段距离后再次检测，如果仍然全灭进入判断，如果不是全灭那就清零重计。

先加大幅度，转到想要的方向，然后通过距离判断转到哪里了，然后给固定占空比直线走过去。

```
        {
            if(Location>19)                         //当累计路程大于19，正式进入丢线，否则算作误判，不进入丢线
            {
            Element_Flag=2;
            Place_Enable=0;
            Place_Out= -100;                        //首先大幅度转向到你想要的方向
            Noline_Flag=2;
            Clear_Location();                       //清零编码器，通过距离来判断转了多少，（用陀螺仪也可以）
            }
        }
        else
            {
                Clear_Location();
                Noline_Flag=0;
            }

f(Noline_Flag==2&&Location>25)      //转到想要的方向后，走直线加速冲过去
{
    Noline_Flag=3;
    PWM_Enable=0;
    Motor_SetPWM_L(6000);                       //设置固定的占空比
    Motor_SetPWM_R(6000);

f(Noline_Flag==3&&Location>=50&&(L2==1||L1==1||M==1||R1==1||R2==1))//识别到线之后，恢复正常寻线
    {

        Noline_Flag=4;
        Element_Flag=0;
        Clear_Location();
    }
```



四、 注意事项

1、如果打开代码发现全是乱码将字体格式调为 GB2312 即可。

2、编码器参数不一定可以直接用，需要自己手推 1m 或者 2m，看一下总共的编码器数值，在实际转换成自己对应的值。

```
Location+=(Speed_L+Speed_R)*Real_Distance;
```

3、如果有各种错误或者更好的意见，欢迎 b 站搜索芋头雨-跟我交流。